Reproducible Research with Git & GitHub

Data Science for the Psychological Sciences

Paul D Kieffaber, Ph.D.

January 28, 2026

Today's goals

Goals:

- Explain what *reproducible research* means (and what it doesn't)
- Use Git for a basic research workflow: **pull** → **edit** → **status** → **add** → **commit** → **push**
- Understand collaboration on GitHub using **branches** and **forks**
- Apply a few best practices that prevent common "Git disasters"

A guiding idea

In computational science, the paper is not the scholarship; it advertises the scholarship.

"An article ... in a scientific publication is not the scholarship itself, it is merely advertising of the scholarship, actual scholarship is the complete software environment and the complete set of instructions which generated the figures."

— *David Donoho*

What is reproducible research?

Reproducibility: an independent person can obtain the *same results* using:

- the **same data**
- the **same analysis code**
- the **same documented steps / environment**

Key idea: reproducibility is about *verifying the original workflow*.

Reproducibility vs generalizability

- **Reproducible**: same procedures $\rightarrow$ same result
- **Generalizable**: result holds in different populations/settings

A study can be reproducible but not generalizable (and vice versa).

A failure to "replicate" often means that research is not generalizable.

Why it matters

When workflows are not reproducible, we risk:

- undetected coding errors
- "invisible" data exclusions or transformations
- incorrect conclusions that spread into policy/practice

A motivating case

The Reinhart and Rogoff Case Study

- The Claim: A highly influential study claimed a strong link between high national debt and low economic growth, shaping global debates.
- The Problem: When graduate students attempted to replicate the findings, they couldn't. The original authors shared their Excel spreadsheet.
- The Discovery: The replication attempt uncovered major errors:
  - Basic coding mistakes in the Excel spreadsheet.
  - Selective exclusion of data.
  - Unconventional weighting of statistics.

Main takeaway: **transparent workflows prevent silent failure.**

What you should share

A reproducible analysis usually includes:

- raw or minimally processed **data** (or a clear access path)
- **analysis scripts** (R/Python/etc.)
- **documentation** (README + instructions)
- environment details (package versions, OS notes, session info)

Where Git fits in

Git is a **version control system** that:

- records changes to files over time
- lets you compare versions and roll back safely
- supports parallel work (branches) and collaboration

Git is a "time machine" for your project:

- commits are snapshots you can return to
- history is auditable (who changed what, when, and why)

Git vocabulary (minimum set)

- **Repository (repo)**: a project folder tracked by Git
- **Commit**: a snapshot + message ("what changed and why")
- **Staging area**: what will go into the *next* commit
- **Branch**: a parallel line of development
- **Remote**: a server copy (e.g., GitHub)

Install + verify Git (quick check)

1. Install Git

- **Windows**: Download the installer from git-scm.com. Use the default settings, especially 'Git from the command line' and 'Checkout Windows-style, commit Unix-style line endings.'
- **macOS**: Open Terminal and run `$ git --version`. If not installed, it will prompt you to install Command Line Developer Tools.

2. Configure Your Identity

Git needs to know who you are to attribute your commits correctly. Run these commands once on any new machine.

```
$ git config --global
user.name "Your Name"
$ git config --global
user.email "your.email@exam
ple.com"
```

3. Basic Terminal Navigation

`pwd` - print working directory (see where you are)

`ls` - list files and folders

`cd <folder_name>` - change directory

`mkdir <folder_name>` - make a new directory

Configure your identity (one-time)

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Verify:

```
git config --global user.name
git config --global user.email
```

Repositories: local vs GitHub

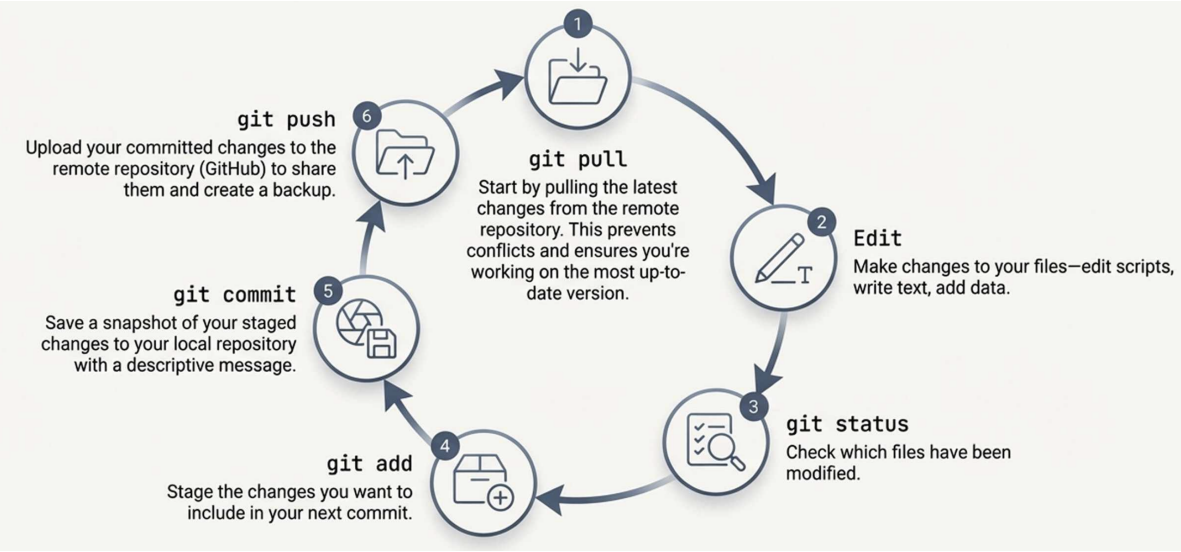
- **Local repo**: lives on your computer
- **Remote repo** (GitHub): your backup + collaboration hub

Best beginner workflow:

1. Create repo on GitHub
2. Clone it locally
3. Work locally
4. Push changes

The basic workflow (the one to memorize)

Pull → Edit → Status → Add → Commit → Push




```
git pull
# edit files
git status
git add .
git commit -m "Describe what changed (and why)"
git push
```

Live demo plan

We will:

1. Create a GitHub repo
2. Clone it
3. Edit `README.md`
4. Check status
5. Commit + push
6. Confirm the change appears on GitHub

Create a GitHub Repo

- Log into GitHub
- Select the “Repositories” tab and click “New”
- Create a *public* repo using the default settings.

Clone the Repo

- Clone the repo to your local computer

```
$ cd SOME_FOLDER
$ git clone LINK_TO_YOUR_REPOSITORY
```

- Navigate to the new local repository

```
$ cd ReproducibleResearch_FIRST_LAST
```

Edit Your Repository

- Add README.md
- Edit files
- etc.

`git status` is your dashboard

```
git status
```

It tells you:

- what changed
- what's staged
- what's untracked

Use it constantly.

Stage changes (`git add`)

Stage **all** changes:

```
git add .
```

Or stage a single file:

```
git add README.md
```

Commit changes (`git commit`)

```
git commit -m "Add course header to README"
```

Commit messages should be:

- specific
- action-oriented
- readable later

Push to GitHub (`git push`)

```
git push origin main
```

Now your commit is on the remote.

Version control for research artifacts

Git works well for:

- analysis scripts (`.R`, `.py`)
- manuscripts (`.Rmd`, `.tex`, `.md`)
- stimuli and experiment scripts (reasonable-sized files)
- documentation (README + notes)

Repository structure (recommended)

A simple pattern:

```
project/
  data/         # often ignored if large or sensitive
  scripts/
  results/      # often ignored if generated
  docs/
  README.md
```

`.gitignore`: prevent tracking the wrong files

Common things to ignore:

- large raw data extracts
- generated outputs (`results/`, logs)
- secrets (API keys)

Example:

```
/results/
/*.log
/data/*.csv
/data/*.tsv
```

Markdown + GitHub (why you care)

Markdown is for:

- READMEs
- lab notes
- simple reports

GitHub renders `.md` automatically, so your documentation becomes readable online.

Markdown basics (quick)

- Headings: `#`, `##`, `###`
- Emphasis: `*italic*`, `**bold**`
- Lists: `- item`
- Code: backticks for inline, fenced blocks for chunks

Appendix: command cheat sheet

```
# status + review
git status
git diff
git diff --cached
git log --oneline --graph --all

# basic workflow
git pull
git add .
git commit -m "message"
git push

# branches
git switch -c new-branch
git switch main
git merge new-branch

# tags
```